Information Sciences 735 (2026) 123110

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Large language model assisted evolutionary neural architecture search with population knowledge base enhancement

Weilin Fang [a], Yu Xue [a,*], Lilian Yuan [a], Mohammad Kamrul Hasan [b], Khursheed Aurangzeb [c]

[a] School of Software, Nanjing University of Information Science and Technology, Nanjing, China
[b] Center for Cyber Security, Faculty of Information Science and Technology, Universiti Kebangsaan Malaysia, Bangi, Malaysia
[c] Computer Engineering Department, College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia

ARTICLE INFO

Keywords:
Surrogate model
Evolutionary algorithm
Large language model
Neural architecture search

ABSTRACT

In the task of evolutionary neural architecture search (ENAS), computational efficiency and resource consumption remain significant bottlenecks. To mitigate these challenges, traditional methods typically employ neural network-based surrogate models to predict the performance of candidate architectures. However, the design and parameter tuning of these surrogate models are heavily reliant on expert knowledge and experience. This paper introduces a novel approach based on large language models (LLMs) to replace conventional machine learning surrogate models, thereby reducing the burden on experts. Although LLMs exhibit powerful zero-shot capabilities, they often lack the domain-specific expertise required for tasks such as neural network architecture evaluation. To overcome this limitation, we propose an LLM-assisted ENAS framework, enhanced by a population knowledge base (PKB). We integrate the LLM as both regression-based and classification-based surrogate models and conduct experiments on NAS-Bench-101, NAS-Bench-201, and the DARTS search space. Our experimental results demonstrate that, with PKB assistance, LLMs can effectively replace traditional surrogate models in evaluating the fitness of neural network architectures. The code is publicly available at: https://github.com/baigeixiaowang/LLM-ENAS-PKB.

1. Introduction

Deep neural networks have achieved groundbreaking advances in various complex domains, such as medical diagnosis, autonomous driving [1], and facial recognition [2]. These achievements are largely driven by continuous innovations in neural network architectures. However, designing high-performance network structures manually is both highly challenging and heavily reliant on the experience of domain experts. With the rise of neural architecture search (NAS), network design has been increasingly automated. Extensive studies have shown that NAS can generate architectures comparable to or even surpassing those designed by human experts, providing more efficient solutions for different application scenarios.

One of the core challenges in NAS lies in the performance evaluation of a large number of candidate architectures, which is often associated with tremendous computational costs. For example, the reinforcement learning-based NAS proposed by Zoph et al. [3] consumed approximately 800 GPUs over the course of one month, while the evolution-based NAS developed by Real et al. [4] employed 250 GPUs running continuously for 11 days. Such resource demands are prohibitively expensive for most researchers. To reduce this cost, recent works have introduced surrogate models that predict the performance of candidate architectures, thereby

* Corresponding author.
Email addresses: fangwl@nuist.edu.cn (W. Fang), xueyu@nuist.edu.cn (Y. Xue), yuanlilian@nuist.edu.cn (L. Yuan), mkhasan@ukm.edu.my (M.K. Hasan), kaurangzeb@ksu.edu.sa (K. Aurangzeb).

https://doi.org/10.1016/j.ins.2026.123110
Received 6 October 2025; Received in revised form 27 December 2025; Accepted 10 January 2026
Available online 12 January 2026
0020-0255/© 2026 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

avoiding costly full training and evaluation. The adoption of surrogate models has reduced NAS hardware requirements from hundreds or even thousands of GPUs to fewer than ten. To improve predictive accuracy, researchers have proposed various neural-network-based surrogate models, such as long short-term memory, graph convolutional networks, LeNet-5, and multilayer perceptrons.

Recent developments in NAS have been rapid, with notable contributions including model-based, supernet-based, and zero-cost prediction-based NAS algorithms. Kadlecova et al. [5] proposed the neural graph feature for predicting neural architecture performance. By combining the neural graph feature with zero-cost proxies and utilizing simple features such as operation counts and node degrees, they achieved strong performance in the architecture evaluation phase. Xiang et al. [6] introduced a neural architecture representation combining graph neural networks and Transformers, significantly improving the accuracy of architecture performance prediction through effective feature fusion and feature selection strategies. Jeon et al. [7] developed a dynamic supernet training method that incorporates complexity-aware learning rate schedules and momentum separation to mitigate fairness and momentum noise issues in N-shot NAS, resulting in more reliable search performance. Oh et al. [8] introduced a new few-shot NAS method, efficiently partitioning the search space by computing the number of nonlinear functions in the subnetwork. Lee et al. [9] proposed AZ-NAS, a training-free method that integrates multiple zero-cost proxies. By combining four new proxies, AZ-NAS captures key architectural characteristics and improves ranking consistency. Additionally, in multi-objective NAS, Aras et al. [10] employed non-dominated sorting genetic algorithms to balance accuracy and hardware-related complexity.

Although significant progress has been made in alternative methods in the architecture evaluation phase of NAS, such as surrogate models, supernets, and zero-cost algorithms, these methods still face a major challenge: the design of network structures, hyperparameter tuning, and the extraction of network expressiveness remain highly complex and heavily reliant on expert knowledge. This contradicts the original goal of NAS, which aims to reduce the reliance on expert design [3]. In addition, the applicability of these methods is often restricted to specific domains, and their transfer to new tasks still requires considerable domain expertise and computational resources. As a result, the need for a general-purpose method to replace the architecture evaluation phase has become a critical bottleneck in the current NAS field.

Recent advances in large language models (LLMs) have introduced new capabilities relevant to architecture evaluation. Research has shown that LLMs possess cross-domain knowledge transfer capabilities, strong reasoning abilities [11], and generalization capabilities in zero-shot learning, and can assist in the construction of fine-grained robotic fault-diagnosis knowledge graphs via ontology refinement and nested NER-based data augmentation [12]. These characteristics enable LLMs to perform expert-level decision-making and cognitive tasks, even without task-specific data. These capabilities motivate the exploration of LLMs as potential surrogates for the architecture evaluation stage of NAS. Based on this motivation, we propose large language model assisted evolutionary neural architecture search with population knowledge base enhancement (LLM-ENAS-PKB). Our main contributions are as follows:

(1) We propose LLM-ENAS-PKB, a framework that incorporates LLM-based surrogates into the evolutionary NAS pipeline for architecture evaluation. The framework replaces conventional surrogate models with LLM inference, reducing the computational overhead of traditional ENAS.
(2) We develop two LLM-based surrogate models—a regression-based surrogate model and a classification-based surrogate model—and compare their performance through controlled experiments, analyzing their respective behaviors in architecture evaluation.
(3) We propose a population knowledge base (PKB) mechanism inspired by retrieval-augmented generation (RAG), featuring a dual-objective elite selection strategy that balances performance and diversity, to provide the LLM with high-quality, relevant historical architecture contexts, thereby significantly reducing the risk of model "hallucinations" and enhancing prediction accuracy in neural architecture evaluation.

2. Related work

This section describes three types of research related to the methodology presented in this paper: Section 2.1 outlines the algorithmic process of ENAS and its related work. Section 2.2 summarizes the applications of LLM in algorithm design. Section 2.3 introduces an algorithmic framework for enhancing LLM applications in vertical domain performance.

2.1. ENAS

NAS is inherently characterized by a two-level optimization scheme. This process involves an inner optimization loop and an outer optimization loop. The inner loop is dedicated to optimizing the weights, denoted as w_α^*, for a specific candidate architecture α. This optimization is achieved by training the architecture on a training set D_{tra} to minimize the training loss. The outer loop's responsibility is to systematically explore a predefined search space Ω_α for potential architectures. After the weights associated with each candidate architecture have been optimized, the outer optimization stage evaluates these architectures on a validation set D_{val} and selects those exhibiting superior performance. In this formulation, $N(\alpha, w)$ corresponds to the neural network constructed by combining an architectural specification α with its parameter vector w. The interaction between architecture selection and weight optimization is described by the following optimization problem:

$$\begin{aligned} w_\alpha^* &= \arg\min_w \mathcal{L}_{\text{tra}}\left(N(\alpha, w), D_{\text{tra}}\right), \quad \alpha \in \Omega_\alpha \\ \alpha^* &= \arg\min_\alpha \mathcal{L}_{\text{val}}\left(N(\alpha, w_\alpha^*), D_{\text{val}}\right). \end{aligned} \tag{1}$$

In this mathematical representation, the first line signifies the optimization of weights w_α^* for a given architecture α by minimizing the training loss. Following this, the second line details the selection of the optimal architecture α^* by minimizing the validation loss, utilizing the previously optimized weights.

ENAS incorporates evolutionary algorithms into NAS in order to achieve both global exploration of architectures and local optimization during the search process. This is typically achieved by maintaining a population of candidate architectures. Each individual architecture within this population first undergoes a process of training and subsequent evaluation to determine its fitness, which is its performance on the validation set. Subsequently, a fitness-based environmental selection mechanism is applied to retain superior individuals. These high-performing individuals then serve as parents for the next generation, undergoing crossover and mutation operations to produce new architectures.

ENAS methodologies leverage the potent global search and iterative local refinement strengths of evolutionary algorithms, thereby significantly enhancing the effectiveness and diversity of automated deep learning architecture design. For instance, Xue et al. [13] introduced a method that dynamically updates surrogate models using multiple pairwise comparisons, enabling efficient filtering of candidate architectures during the evolutionary process and substantially reducing training and evaluation overhead. In another contribution [14], they demonstrated notable improvements in search efficiency and the final architecture performance by implementing a dual encoding strategy for both network architecture and its performance, which in turn guides surrogate model evaluation based on the similarity between candidate and already evaluated architectures. Recently, Huang et al. [15] proposed an ENAS framework that integrates an elite weight inheritance mechanism. In this approach, newly generated architectures are initialized by inheriting parameters from high-performing individuals in previous generations, which effectively accelerates the convergence of the evolutionary process while retaining valuable historical information accumulated during the search.

2.2. Large language models

In recent years, LLMs have shown strong performance in reasoning, text generation, and knowledge transfer across natural language and multimodal tasks. Owing to their strong in-context learning and pattern extraction abilities, LLMs excel not only in traditional text generation, but also in algorithm design, combinatorial optimization, and continuous optimization. Particularly, when combined with evolutionary algorithms (EAs), LLMs can function in multiple roles within the search process, thereby improving both optimization efficiency and solution quality.

2.2.1. LLMs as combinatorial optimizers

In combinatorial optimization tasks, LLMs can be regarded as black-box optimizers that generate and refine candidate solutions within the search space. Through in-context learning, LLMs can iteratively adjust candidate solutions based on previously observed patterns. When integrated with EAs, LLMs can act as auxiliary search operators, contributing to exploration and exploitation during optimization. For example, in complex scheduling, path planning, or resource allocation problems, LLMs can directly generate high-quality feasible solutions from problem descriptions and continuously improve them during iterations [16]. Recent NAS studies also explore the use of LLMs as combinatorial optimizers. SEKI [17] treats the LLM as the primary search engine that iteratively refines architectures through self-evolution and distilled design patterns, achieving competitive architectures without expensive full training. RZ-NAS [18] integrates zero-cost proxies and reflective feedback, enabling the LLM to perform high-level structural revisions that guide the search toward stronger architectures with improved stability and efficiency.

2.2.2. LLMs as feature extractors

LLMs can learn representations from problem descriptions, source code, and structured data. This capability is particularly valuable for high-dimensional and structurally complex optimization problems, where conventional feature engineering is challenging. When integrated with EAs, an LLM can serve as an encoder, mapping solutions from the original search space into meaningful semantic or low-dimensional vector representations, thus improving the efficiency of search and optimization processes. In domains such as wireless resource allocation and urban building recognition, this representation capability has been shown to significantly enhance both the accuracy and computational efficiency of problem-solving. In NAS, recent work has explored the use of LLMs as feature extractors. STS [19] uses LLMs to extract task-structure relations from spatiotemporal data and candidate architectures. The extracted features guide subsequent search operations.

2.2.3. LLMs as predictors

In the EA-enhanced framework, LLMs can serve as surrogate models to evaluate or predict the performance of candidate solutions. This prediction process can be formulated as a classification or regression task, where the LLM approximates the mapping from a solution to its objective value. This surrogate modeling approach is particularly useful in optimization problems with high computational costs. For example, LLMatic [20] utilizes the encoding capability of LLMs to introduce meaningful variations in defining the code for neural networks. It generates competitive networks after evaluating only 2000 candidate networks. Similarly, GENIUS [21] uses GPT-4 as a black-box optimizer to efficiently navigate the architecture search space, identify promising candidates, and iteratively refine these candidates to enhance performance. MONAQ [22] introduces an evaluation agent that, through prompt engineering, enables LLMs to perform training-free performance prediction and ranking under multi-objective constraints, effectively replacing traditional surrogate models. SCD-NAS [23] employs an LLM as a zero-cost predictor to estimate candidate performance and combines search space expansion techniques to integrate architectural advantages, improving the stability and effectiveness of architecture selection without requiring costly training-based assessments.

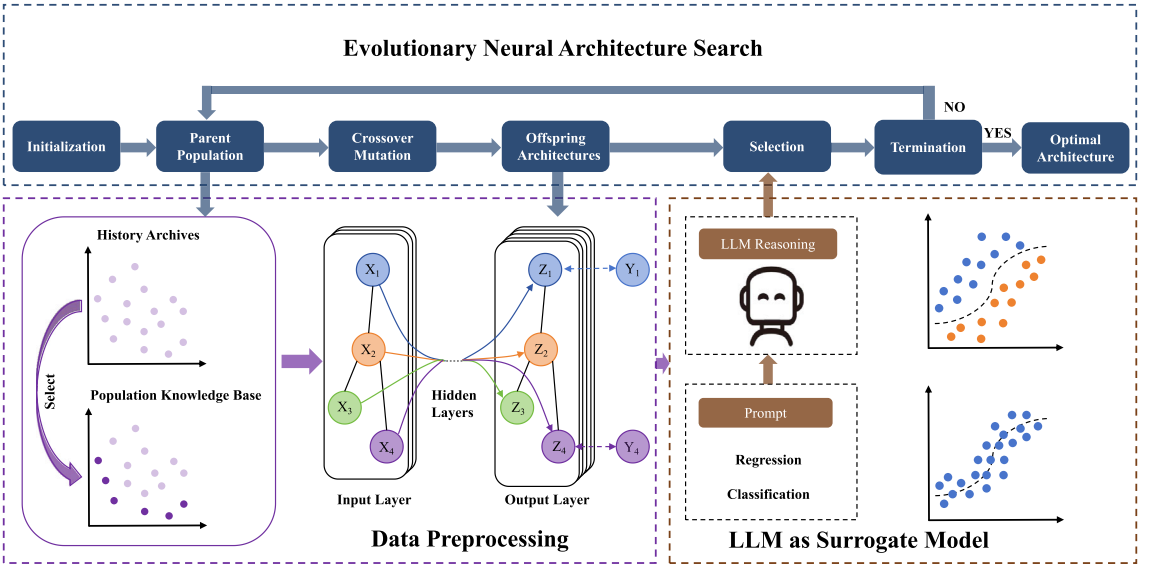


Fig. 1. An overview of the LLM-ENAS-PKB framework.

While these methods are pioneering, they overlook the issue of knowledge scarcity in LLMs for domain-specific ENAS tasks. As a result, their search cost and accuracy fall short when compared to traditional surrogate methods in the expert-designed architecture evaluation stage. To achieve this, we propose an algorithmic framework that enhances LLMs using the PKB to overcome the knowledge gaps in the vertical domain. Our experimental results demonstrate that after the enhancement with PKB, the performance of LLMs significantly improves, achieving comparable results to traditional expert-designed surrogate methods.

2.3. Retrieval-augmented generation

RAG [24] is an architecture that integrates retrieval mechanisms with generative models to enhance the knowledge utilization and reasoning capabilities of LLMs. Unlike conventional generation approaches that rely solely on parameterized knowledge stored within the model, RAG incorporates an external knowledge base during the generation process. It retrieves relevant documents in real time and integrates them with the generative component to produce informed outputs, thereby addressing the limitations of LLMs in specialized knowledge, dynamic information access, and complex reasoning.

The necessity of RAG arises from the static nature of LLM training corpora and the inherent lag in knowledge updates, which can lead to inaccurate responses when encountering newly emerged knowledge or domain-specific queries. By incorporating retrieval, RAG enables LLMs to transcend the limitations of parameter-bound information, dynamically acquiring up-to-date external data, thus significantly improving adaptability and domain expertise. Compared to supervised fine-tuning, RAG offers the advantage of eliminating the dependency on large-scale annotated datasets for task-specific adaptation. Instead, it supplements the model's inference process with external knowledge retrieved at runtime, granting greater flexibility and generalization when addressing unseen tasks or cross-domain scenarios. In highly specialized domains such as medicine and law, RAG is particularly critical, as it enables the model to access authoritative and timely domain resources during content generation, ensuring both accuracy and reliability. This integration makes RAG a useful way to improve LLM performance in domain-specific applications.

Inspired by the RAG framework, we believe that in the vertical domain task of architecture evaluation for LLM-assisted ENAS, it is also necessary to enhance the knowledge of LLMs. To address this, we integrate the concept of population from EA and propose a framework based on a PKB. This framework enhances the LLM's prompts by supplementing them with domain-specific knowledge, enabling the model to perform more effectively in the task of architecture evaluation.

3. Methodology

This section presents the technical realization of the proposed LLM-ENAS-PKB approach. Specifically, Section 3.1 focuses on the algorithmic structure of the proposed method. Section 3.2 describes the specific implementation of using LLMs as regression-based and classification-based surrogate models. Section 3.3 provides an in-depth discussion of the design of the PKB. Section 3.4 introduces the design of the architecture information encoder.

3.1. Overall framework

As illustrated in Fig. 1, the proposed framework integrates an LLM as a surrogate model to assist in ENAS. The upper part of the figure depicts the execution flow of a conventional ENAS algorithm: the process begins with the initialization of a population to

generate the first generation of parent architectures, which are stored in a historical archive. Subsequently, the parent population undergoes genetic operators such as crossover and mutation to produce a large number of offspring architectures. In traditional approaches, this is followed by a costly evaluation stage in which the algorithm evaluates the offspring and selects the best architectures to form the next generation.

To improve selection efficiency and reduce computational cost, our method introduces an LLM to participate in the architecture evaluation and selection process (lower part of Fig. 1). Specifically, as shown in the lower-left segment, during each offspring selection phase, a representative and diverse subset of architectures is first selected from the historical archive according to a defined sampling strategy, forming the PKB. The PKB construction and selection mechanism is detailed in Section 3.3. Next, both the architectures in the PKB and the newly generated offspring are input into the architecture information encoder, which produces standardized structural representations suitable for LLM consumption, as described in Section 3.4.

As illustrated in the lower-right segment of the framework, the encoded information of the PKB and the offspring architectures is integrated into a unified prompt. Depending on the task type (regression or classification), we construct either a regression-style prompt or a classification-style prompt, which constrains the reasoning behavior of the LLM so that, in regression tasks, it outputs predicted performance values, and in classification tasks, it assigns performance categories. Finally, guided by the constructed prompt, the LLM performs rapid inference and filtering over all offspring candidates. The resulting predictions enable efficient identification of the most promising offspring architectures under limited computational resources. The operational details of using the LLM as a surrogate model are provided in Section 3.2.

3.2. LLM-guided ENAS

The ability of LLMs to understand and evaluate neural network architectures is largely attributed to the domain knowledge acquired during pre-training. LLMs are typically trained on large-scale, multimodal corpora, which include vast amounts of programming code, academic papers, and technical documentation. These corpora cover neural network concepts (e.g., convolutional layers, attention mechanisms), architectural design patterns (e.g., ResNet, Transformer), and performance evaluation metrics (e.g., accuracy, FLOPs). This pre-training knowledge is directly relevant to the architecture evaluation task. The task can be viewed as a nonlinear mapping from structural features (e.g., layer count, connectivity, operation types) to performance metrics (e.g., classification accuracy). The pattern recognition and relational reasoning capabilities of LLMs, supported by the attention mechanisms of the Transformer architecture, allow them to model such complex relationships. Although LLMs can evaluate neural network architectures to some extent, their prediction reliability improves when domain-specific information is incorporated into the prompts.

In this study, we regard the LLM as a surrogate model, functioning as a black-box optimizer to replace traditional machine learning models or surrogate neural networks meticulously designed by researchers. In traditional NAS, surrogate models are typically categorized into two predictive modes: one that directly regresses the performance metrics of architectures, and the other that conducts binary or multi-class classification to evaluate the superiority or inferiority of architectures. These two approaches correspond to different task objectives and usage scenarios, respectively. Compared with conventional approaches, leveraging LLMs pre-trained on large-scale datasets as surrogate models can substantially reduce the workload associated with network architecture design, model training, and hyperparameter tuning. Following the predictive paradigm of traditional surrogate models, we further investigate the feasibility and effectiveness of employing LLMs as both regression-based surrogate models and classification-based surrogate models.

3.2.1. Regression-based surrogate model

As illustrated in Algorithm 1, the LLM-ENAS-PKB framework follows the fundamental search mechanism of conventional ENAS while incorporating targeted modifications in key stages. Lines 1–5 generate a set of candidate offspring architectures through crossover and mutation, lines 6–11 perform data preprocessing for LLM input, and lines 12–22 replace the training-based surrogate evaluation module with a direct performance inference mechanism based on the LLM. At the beginning of each generation, Algorithm 2 is invoked to select, from the historical archive A_{hist}, a set of architectures that possess both high informational value and substantial structural diversity, forming the PKB. This ensures broad coverage of the performance distribution while reducing redundancy, thereby providing high-quality contextual input to significantly enhance the reliability of LLM predictions. The selected historical architectures, together with newly generated candidates, are processed by a training-free architecture information encoder, which integrates network topology and operator-level details into a unified serialized representation, enabling the LLM to efficiently parse and perform reasoning.

As shown in Fig. 2, these encoded representations are embedded into a carefully designed prompt template containing an explicit task description, strict constraints on the reasoning process, reference exemplars with ground-truth performance labels, encoded information of the candidate architectures requiring prediction, and fully specified constraints on both output format and reasoning procedure. The task description specifies the prediction objective and performance metrics, the reasoning constraints promote consistent inference behavior across invocations, and the reference exemplars use in-context information to improve stability in domain-specific prediction. The candidate architecture information omits performance labels to guide independent inference, while the output constraints define acceptable numerical ranges, required precision, and parsing rules to maintain both semantic and numerical comparability across generations. To further mitigate the risk of invalid outputs or formatting errors, multiple fallback strategies are integrated into the prompt design, and an automated validation and exception-handling mechanism is executed after inference. The constructed prompt is finally submitted to the LLM, which directly produces a predicted performance value for each candidate architecture; these predicted values are treated as fitness scores for ranking. The top P architectures are then selected as

Algorithm 1 LLM-Guided ENAS.

Input: $\mathcal{P}_0$: initial parental population; $\mathcal{A}_{hist}$: historical architecture archive;
$\mathcal{K}_{pop}$: population knowledge base (PKB); $\mathcal{A}_{offspring}$: candidate offspring set;
LLM: large language model; O: task type (regression/classification);
$T_{\max}$: maximum number of iterations
Output: A^*: discovered optimal architecture

1: **for** $t = 1$ to $T_{\max}$ **do**
2: `// Archive all architectures from the current generation`
3: $\mathcal{A}_{hist} \leftarrow \mathcal{A}_{hist} \cup \mathcal{P}_t$
4: `// Generate candidate offspring via genetic operators`
5: $\mathcal{A}_{offspring} \leftarrow$ `CrossoverMutationOperator`$(\mathcal{P}_t)$
6: `// Select informative architectures from the archive to form PKB (Section 3.3)`
7: $\mathcal{K}_{pop} \leftarrow$ `SelectRelevantArchitectures`$(\mathcal{A}_{hist})$
8: `// Encode PKB architectures into LLM-readable representations (Section 3.4)`
9: $\mathcal{K}_{enc} \leftarrow$ `ArchitectureInformationEncoder`$(\mathcal{K}_{pop})$
10: `// Encode offspring architectures into LLM-readable representations (Section 3.4)`
11: $\mathcal{A}_{offspring_enc} \leftarrow$ `ArchitectureInformationEncoder`$(\mathcal{A}_{offspring})$
12: `// Generate unified prompt for LLM input (Section 3.2)`
13: `Prompt` $\leftarrow$ `GeneratePrompt`$(\mathcal{K}_{enc}, \mathcal{A}_{offspring_enc}, O)$
14: **if** $O =$ `Regression` **then**
15: `// Predict performance scores using LLM`
16: $\mathcal{R}_{offspring} \leftarrow$ `PredictPerformance`$(LLM,$ `Prompt`$)$
17: `// Sort offspring based on predicted scores`
18: $\mathcal{S}_{offspring} \leftarrow$ `SortByScore`$(\mathcal{R}_{offspring}, \mathcal{A}_{offspring})$
19: **else**
20: `// Classify offspring performance using LLM`
21: $\mathcal{S}_{offspring} \leftarrow$ `ClassifyPerformance`$(LLM,$ `Prompt`$)$
22: **end if**
23: `// Perform tournament selection to generate next generation population`
24: $\mathcal{P}_{t+1} \leftarrow$ `TournamentSelection`$(\mathcal{P}_t \cup \mathcal{S}_{offspring})$
25: **end for**
26: **return** A^*

the offspring population for the next generation, enabling efficient architecture performance evaluation without the need to train a surrogate model.

3.2.2. Classification-based surrogate model

In the case where an LLM is employed as a classification-based surrogate model, the core workflow largely aligns with that of the regression task. For consistency, both the regression and classification settings use the same LLM backbone, and their differences appear only in a few task-specific stages. As shown in Algorithm 1, these differences mainly occur between lines 13–22, namely in the prompt construction, LLM prediction output, and post-processing steps.

As shown in Fig. 3, the prompt for the classification task follows the structure of the regression prompt and consists of five components: a task description, reasoning step constraints, labeled reference examples, information on candidate architectures for prediction, and output content with reasoning restrictions. However, in terms of specific content, the classification prompt explicitly states, in both the task description and the reasoning step instructions, that the current task is a binary classification problem. The objective is to determine whether the architecture's performance belongs to the "high-performance" or "low-performance" category, instead of predicting a specific continuous performance value.

In the reference examples, the labels for the classification task are no longer precise performance values but binary labels taking values from $\{1, 0\}$, where 1 represents a high-performance architecture and 0 represents a low-performance architecture. The classification threshold is determined based on the median performance in the PKB: architectures with performance above the median are assigned label 1, and those below the median are assigned label 0. In terms of output specification, the classification task constrains the prediction target of the LLM to output only category labels 1 or 0. In line 14 of Algorithm 1, the conditional judgment logic is used to differentiate between the post-processing strategies for regression and classification: for regression, it is necessary to perform a global ranking based on predicted performance values to select the candidate architectures with the highest fitness; for classification, post-processing directly filters out architectures predicted by the LLM with label 1 (high-performance), which are then retained as the offspring population for the next generation.

From the perspective of traditional machine learning, binary classification is generally considered easier to learn than regression, as the task objective is more explicit and the label space is smaller. However, when the surrogate model is an LLM, this assumption may not necessarily hold. Although LLMs can learn classification boundaries from in-context examples and thus rely less on diverse

Algorithm 2 Performance-Diversity Based Elite Selection.

Input: P: Population, each individual is $(architecture_vector, performance_value)$;
α: performance score weight $(0 \leq \alpha \leq 1)$;
K_{perf}: number of individuals selected based on performance-diversity combined score;
K_{div}: number of individuals selected based on crowding distance;
$selected_size$: final number of selected individuals
Output: $\mathcal{K}_{pop}$: set of selected elite individuals

`// Step 1: Map architecture vectors into a high-dimensional embedding space`
for each $(arch, val)$ **in** P **do**
  $embeddings \leftarrow$ `ArchitectureEmbedding`$(arch)$
end for
`// Step 2: Normalize performance values to range [0, 1]`
$perf_scores \leftarrow$ `Normalize`$([val$ **for each** $individual$ **in** $P])$
`// Step 3: Compute similarity matrix between architectures`
$sim_matrix \leftarrow$ `CosineSimilarity`$(embeddings)$
`// Step 4: Compute combined score (performance + diversity)`
for $i = 1$ **to** $|P|$ **do**
  $diversity_score \leftarrow 1 - Mean(sim_matrix[i])$
  $score[i] \leftarrow \alpha \cdot perf_scores[i] + (1 - \alpha) \cdot diversity_score$
end for
`// Step 5: Compute crowding distance in performance-diversity space`
$crowd_distances \leftarrow$ `CrowdingDistance`$(perf_scores, sim_matrix)$
`// Step 6: Select top` K_{perf} `individuals by combined score`
$top_perf_idx \leftarrow$ `TopK`$(score, K_{perf})$
`// Step 7: Select top` K_{div} `individuals by crowding distance`
$top_div_idx \leftarrow$ `TopK`$(crowd_distances, K_{div})$
`// Step 8: Get globally sorted indices by descending combined score`
$global_sorted_idx \leftarrow$ `SortDescending`$(score)$
`// Step 9: Merge performance-based and diversity-based selections, fill with global ranking if needed`
$selected_idx \leftarrow$ `MergeUnique`$(top_perf_idx,\ top_div_idx,\ \max_size = selected_size,\ filler = global_sorted_idx)$
`// Step 10: Return selected elite individuals`
$\mathcal{K}_{pop} \leftarrow \{P[i] \mid i \in selected_idx\}$
return $\mathcal{K}_{pop}$

Regression Task:
You need to predict the target value (Value) for given Candidate Architectures (Architecture Information) based on Population Knowledge Base. This task involves inferring the behavior of a black-box function and understanding the non-linear relationships between attributes and their corresponding values.
.....

Steps:
1. Analyze the Population Knowledge Base to uncover relationships between Architecture Information and Value.
2. Based on these insights, predict the approximate Value for the given Input (Architecture Information).
3. Ensure the prediction follows the specified range and format.

Population Knowledge Base:
<Architecture Information>: [9.3596, 21.6096 ... 9.8364]; <Value>: 0.921246;
.....
<Architecture Information>: [19.1047, 9.8156 ... 4.8025]; <Value>: 0.891439;

Candidate Architectures:
<Architecture Information>: [4.2453, 32.1256 ... 5.4512]; <Value>: ;
.....
<Architecture Information>: [32.4521, 6.4154 ... 8.1537]; <Value>: ;

Constraints:
1. Output must be in valid JSON format,
  Example outputs:
    {{"Value": 0.926452}}
    .....
    {{"Value": 0.875612}}
2. Value range: 0.00 ≤ Value ≤ 1.00.
3. Precision: Value should have at most six decimal places.
.....

Fig. 2. An example of an LLM serving as a regression-based surrogate model for the ENAS task.

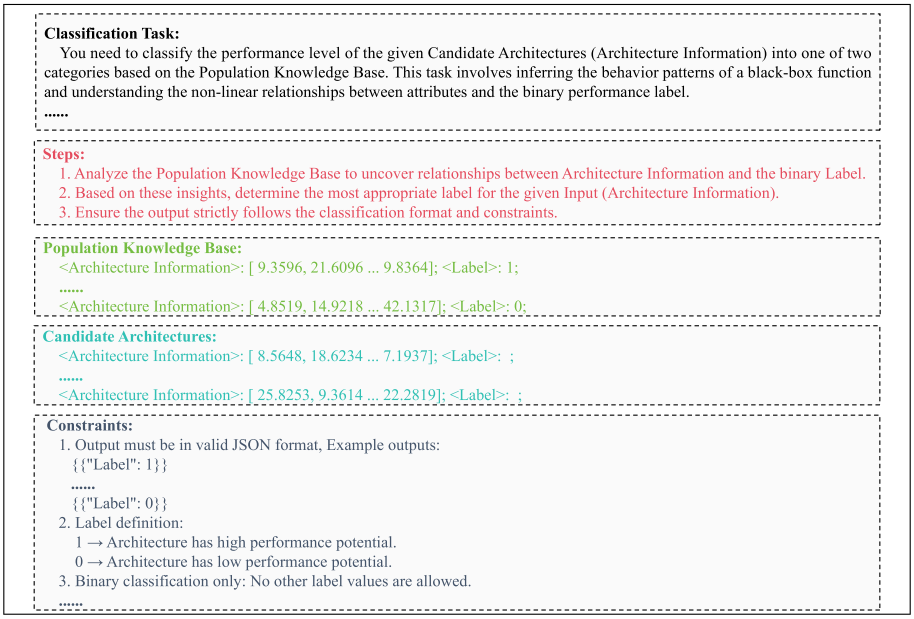


Fig. 3. An example of an LLM serving as a classification-based surrogate model for the ENAS task.

regression data, the lower information content of binary labels may limit their ability to capture fine-grained performance differences. This limitation can reduce the accuracy of selecting optimal architectures. Therefore, it is necessary to conduct experiments to verify the prediction performance of LLMs in classification tasks, and to analyze their differences from regression tasks in terms of search efficiency, stability, and the performance of the final best-found architecture.

3.3. Population knowledge base design

Based on existing research on the RAG framework [24], it is evident that while LLMs possess a wealth of general knowledge, they still struggle to achieve optimal results in vertical domains. Therefore, to enhance the accuracy of architecture evaluation in the ENAS task, it is crucial to provide LLMs with high-quality historical architecture information within the prompt. However, directly including all historical architecture information may lead to redundancy and context overload. When a large volume of similar or irrelevant architectural information is provided to the LLM, the excessive context length can result in unstable behavior, such as hallucinations or information disruptions. If historical architecture samples are selected solely based on performance rankings, the diversity of the search space may be compromised, limiting the identification of architectures with globally optimal performance. On the other hand, if diversity is overly emphasized, low-performance architectures may be introduced, which could mislead the LLM, as ENAS predominantly generates high-performance architectures in later stages. To address these challenges, the primary goal of this study is to strike a balance between the representativeness of architecture performance and the structural diversity of the population, while providing the most valuable architectural information within the limited context length of the LLM. To achieve this, we have designed the PKB and developed a selection strategy to ensure that the most effective information is conveyed within the constrained context. The specific selection strategy for the PKB is outlined as follows:

Each candidate architecture is represented as a 35-dimensional real-valued feature vector, encoding key attributes such as network topology, parameter size, layer depth, and operator types: $\mathbf{x}_i \in \mathbb{R}^{1\times 35}$, where i indexes the individual in the current population. Each architecture has an associated true performance value V_i, determined by the evaluation metric of the specific task, such as classification accuracy, F1-score, or inference latency: $V_i \in \mathbb{R}$. The historical archive of architectures and their performance values is stored in the PKB as: $\mathcal{P} = \{(\mathbf{x}_1, V_1), (\mathbf{x}_2, V_2), \ldots, (\mathbf{x}_N, V_N)\}$, where N is the number of stored individuals. To measure structural distinctiveness between architectures, each $\mathbf{x}_i$ is first mapped to a d-dimensional embedding space using the architecture information encoder: $\mathbf{z}_i = \text{Encoder}(\mathbf{x}_i), \quad \mathbf{z}_i \in \mathbb{R}^d$, where $\mathbf{z}_i$ is the embedding of architecture i, and d is the embedding dimension. The cosine similarity between any two architectures is then computed as:

$$S_{\text{sim}}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{z}_i \cdot \mathbf{z}_j}{\|\mathbf{z}_i\| \cdot \|\mathbf{z}_j\|}. \tag{2}$$

Architectures with a lower average similarity to others are more structurally unique and contribute more to diversity. Cosine similarity is used because it captures directional differences in high-dimensional architecture embeddings and is robust to scale variations. Euclidean distance becomes less discriminative due to the concentration-of-measure effect, making cosine similarity a more reliable metric for structural diversity. Since tasks may have differing performance ranges, all performance values are normalized to the

range $[0, 1]$:

$$S_{\text{perf},i} = \frac{V_i - V_{\min}}{V_{\max} - V_{\min}}, \tag{3}$$

where $V_{\max}$ and $V_{\min}$ are the maximum and minimum performance values in the current PKB. To jointly evaluate the performance and structural diversity of each architecture, a weighted combined score is computed for each individual:

$$\text{Score}_i = \alpha \cdot S_{\text{perf},i} + (1 - \alpha) \cdot \left(1 - \overline{S_{\text{sim},i}}\right), \tag{4}$$

where $\overline{S_{\text{sim},i}}$ denotes the mean cosine similarity between architecture i and all other architectures in the current population, and $\alpha \in [0, 1]$ is the trade-off coefficient. The normalized performance score $S_{\text{perf},i}$ reflects the true performance of the architecture, while the diversity term $1 - \overline{S_{\text{sim},i}}$ measures the average structural difference between the given architecture and the rest of the population. The coefficient α controls the relative importance of performance and diversity: a larger α biases the selection toward high-performing architectures, whereas a smaller α places more emphasis on structural diversity.

To avoid overly clustering selected individuals in the performance–diversity space, the crowding distance is calculated to quantify the sparsity of each solution:

$$D_i = \sum_{m \in \{\text{Perf},\text{Div}\}} \frac{f_m(P^m_{i+1}) - f_m(P^m_{i-1})}{f^{\max}_m - f^{\min}_m}, \tag{5}$$

where P^m_{i+1} and P^m_{i-1} represent the individuals adjacent to individual i when the population is sorted by objective m. $f_m(\cdot)$ is the objective value for objective m. $f^{\max}_m$ and $f^{\min}_m$ are the maximum and minimum values for objective m in the current population, respectively. Here, Perf is represented by the normalized performance score $S_{\text{perf},i}$, and Div is represented by the inverse mean similarity $1 - \overline{S_{\text{sim},i}}$. Higher D_i values indicate that the individual is located in a sparser region of the search space, improving exploration coverage. In our formulation, the crowding distance is computed jointly based on the normalized performance scores and the inverse embedding-based similarities, ensuring that diversity is promoted along both the performance dimension and the structural-representation dimension. Higher crowding-distance values indicate that the corresponding individuals lie in sparse regions of the search space, which enhances global exploration.

Based on these metrics, the elite selection process proceeds in three stages: (1) select the top K_{perf} individuals by Score_i; (2) select the top K_{div} individuals by D_i; (3) merge these sets, remove duplicates, and fill any remaining slots using the global Score ranking until reaching $selected_size$. The resulting set P_{selected} is stored in the PKB and can be provided to the LLM as a compact, representative, and diverse knowledge subset. This method offers several advantages. It achieves a dynamic balance between performance and structural diversity, avoiding the pitfalls of selecting only similar high-performance models or diverse low-performance ones. By combining the score and crowding distance metrics, it ensures that the selected architectures are both strong candidates and provide broad search space coverage. Furthermore, by outputting a refined selection, it maximizes the use of the LLM's limited context length, reducing redundancy while retaining essential search knowledge.

3.4. Architecture information encoder

In this work, the architecture of a neural network is modeled as a directed acyclic graph denoted by $G = (V, E)$, where $V = \{v_1, v_2, \ldots, v_n\}$ represents the set of nodes and $E = \{e_1, e_2, \ldots, e_m\}$ denotes the set of edges. Depending on how the search space is defined, nodes may correspond to computational operators or intermediate feature maps, while edges describe either the data flow or the applied operations. Each edge in E is associated with an operation type, which is first encoded as a one-hot matrix $O \in \{0, 1\}^{m \times l}$, where l is the number of possible operation types. To obtain dense learnable features, this matrix is projected into a d-dimensional feature space through a linear mapping $W \in \mathbb{R}^{l \times d}$ as follows:

$$E_{\text{op}} = OW, \tag{6}$$

where $E_{\text{op}} \in \mathbb{R}^{m \times d}$ contains the embedded edge features.

The initial node representations are derived from one-hot encodings. To incorporate topological information, an iterative neighborhood aggregation mechanism is applied, where each node updates its representation by combining its own features with those of its neighbors, normalized by their degrees. The propagation rule is expressed as

$$h_v^{(k+1)} = h_v^{(k)} + \sum_{u \in N(v)} \frac{h_u^{(k)}}{d_u}, \tag{7}$$

where $h_v^{(k)}$ is the representation of node v at iteration k, $N(v)$ denotes the set of neighboring nodes of v, and d_u is the degree of node u (sum of in- and out-degrees). In our experiments, we set $k = 3$ so that each node can capture information from higher-order neighbors.

After the final iteration, the nodes are arranged according to a valid topological ordering of G to ensure dependency consistency. The ordered node features $\{h_v^{(K)}\}_{v \in V}$ are then concatenated and flattened to form a fixed-length vector representation $\mathbf{x} = \text{Flatten}(\text{TopoSort}(\{h_v^{(K)}\}_{v \in V}))$.

Table 1
The experimental parameter configurations utilized for NAS-Bench-101, NAS-Bench-201, and DARTS.

	Parameter	NAS-Bench-101	NAS-Bench-201/DARTS
LLM	d: Input dimension	35	30
	Model	DeepSeek-R1	DeepSeek-R1
	Temperature	0.3	0.3
	Top-p	0.8	0.8
	Repetition_penalty	1.8	1.8
Evolution	Maximum number of iterations	30	30
	Population size	20	20
	Crossover probability	0.8	0.8
	Mutation probability	0.2	0.2
	k: Aggregation depth	3	3
PKB	α: Performance score weight	0.6	0.6
	K_{perf}: Performance-diversity combined score	$\{4, 5, \ldots, 30\}$	$\{4, 5, \ldots, 30\}$
	K_{div}: Crowding distance	$\{12, 13, \ldots, 30\}$	$\{12, 13, \ldots, 30\}$
	$selected_size$: Final number of selected individuals	$\{20, 21, \ldots, 30\}$	$\{20, 21, \ldots, 30\}$

4. Experimental results

4.1. Datasets and experimental setup

This work uses two representative NAS benchmarks—NAS-Bench-101 [25] and NAS-Bench-201 [26]—together with the large-scale DARTS search space [27], to enable a systematic and reproducible evaluation of LLM-ENAS-PKB. The NAS-Bench suites provide exhaustive search spaces with pre-computed training results, enabling controlled experimental comparisons without the cost of training every candidate architecture. The inclusion of the DARTS search space further allows assessment of the method's behavior in a more realistic and substantially larger design space.

1. **NAS-Bench-101:** This benchmark explores a relatively large search space with over 420K candidate architectures, each described as a directed acyclic graph. The architectures are cell-based, and each cell contains a set of possible operations connected by edges to define computation paths. All networks are trained on CIFAR-10 under a unified protocol, enabling direct comparison of architecture performance.
2. **NAS-Bench-201:** In contrast, this benchmark focuses on a smaller but more structured search space containing 15,625 models. Its encoding scheme differs by using nodes to represent feature maps and edges to represent candidate operations. A key advantage of NAS-Bench-201 is the availability of performance metrics on multiple datasets (CIFAR-10, CIFAR-100, ImageNet-16–120), which allows researchers to study generalization behavior across different tasks.
3. **DARTS:** This search space uses a cell-based design composed of Normal Cells and Reduction Cells. Each cell contains multiple intermediate nodes, and the directed edges between nodes can choose from eight candidate operations, including various convolution, pooling, and identity mappings. Normal Cells keep the spatial resolution unchanged, whereas Reduction Cells downsample feature maps using stride operations. The final network is formed by stacking these two types of cells. Since each edge independently selects an operation, the overall search space reaches the order of 10^{21} possible architectures, making it suitable for evaluating the scalability and robustness of NAS methods in large search spaces.

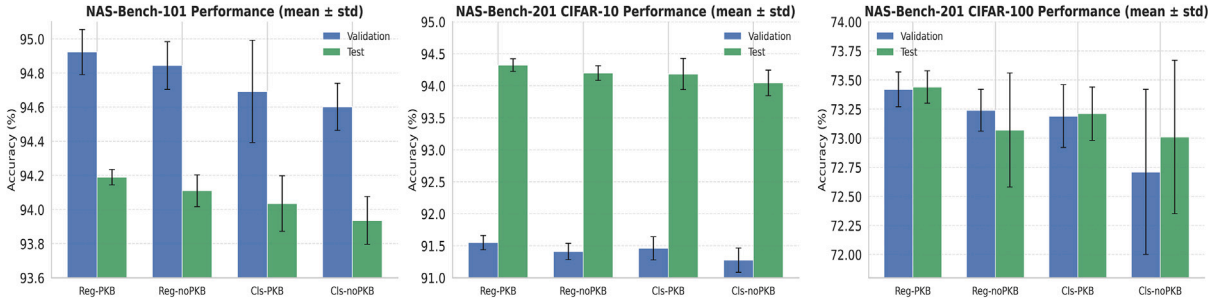
Table 1 provides a summary of the hyperparameter configurations employed for the LLM, evolutionary search, and PKB screening processes. As shown in Table 1, the hyperparameters used in this work play critical roles in stabilizing LLM-based inference, guiding the global exploration behavior of the evolutionary algorithm, and ensuring the quality and representativeness of the PKB. For the LLM module, the input dimension ensures a unified architecture encoding, the temperature and top-p parameters regulate the stochasticity and diversity of generation, thereby improving prediction stability, and the repetition_penalty suppresses repetitive patterns to reduce inference bias. In the Evolution module, the population size, maximum iteration number, and crossover/mutation probabilities determine the balance between exploration and exploitation during the search. For the PKB module, the performance score weight, performance–diversity combined score range, and crowding distance work jointly to retain representative historical architectures within the LLM's limited context window, improving prediction accuracy and mitigating hallucinations. Moreover, several hyperparameters in Table 1 are intrinsically linked to the structural encoding process defined in Eq. (7). The input dimension d (e.g., 35 or 30) arises from the distinct operator-placement rules and feature compositions of NAS-Bench-101 and NAS-Bench-201, respectively, while the aggregation depth $k = 3$ follows established findings in graph neural network literature indicating that 2–3 iterations of message passing are sufficient to capture essential structural dependencies without incurring oversmoothing. These clarifications ensure that the architecture representations used for LLM-based inference remain consistent and stable across different benchmark settings. A detailed sensitivity analysis of these hyperparameters is provided in Section 4.5.5.

4.2. Results on NAS-bench-101

As shown in Table 2, we evaluate the proposed LLM-ENAS-PKB method against several state-of-the-art (SOTA) NAS algorithms within the NAS-Bench-101 search space. The comparison metrics include the number of architecture evaluations performed during

Table 2
Results comparing LLM-ENAS-PKB and other NAS methods on NAS-Bench-101. "-" means unavailable results.

Algorithms	#Queries	Accuracy (%)	Ranking(%)
Oracle	N/A	94.32	0.0002
NAR (statistics) [28]	4236	94.07 ± 0.09	0.0057
NAR (random) [28]	4236	94.06 ± 0.04	0.0064
DARTS [27]	–	92.21 ± 0.61	19.6133
SSENAS [14]	1000	93.83 ± 0.14	0.0857
LaNAS [29]	1000	94.10	0.0033
BONAS [30]	1000	94.22	0.0007
SMCSO [13]	734	94.15 ± 0.06	0.0017
BANANAS [31]	500	94.08	0.0050
ERNAS [32]	423	93.94 ± 0.08	0.0253
Arch2vec [33]	400	94.10	0.0032
Reg-PKB	**400**	**94.19 ± 0.05**	**0.0011**
Reg-NoPKB	400	94.11 ± 0.09	0.0023
Cls-PKB	400	94.04 ± 0.16	0.0077
Cls-NoPKB	400	93.94 ± 0.14	0.0231



(a)NAS-Bench-101 (b)NAS-Bench-201(CIFAR-10) (c)NAS-Bench-201(CIFAR-100)

Fig. 4. The figure shows three paneled grouped bar charts for NAS-Bench-101, NAS-Bench-201 CIFAR-10, and NAS-Bench-201 CIFAR-100, where the x-axis lists methods (Reg-PKB, Reg-NoPKB, Cls-PKB, Cls-NoPKB) and the y-axis reports accuracy; blue and green bars denote validation and test performance, respectively. Each bar gives the mean over repeated runs, and the vertical error bars indicate the standard deviation, reflecting run-to-run variability.

the search (#Queries), the average classification accuracy on the test set (Accuracy), and the percentile ranking (Ranking) within the entire search space, where a smaller ranking value indicates that the obtained architecture is closer to the global optimum. The "Oracle" row in the table provides the theoretical upper bound corresponding to the optimal architecture. To reduce the impact of stochastic variance, all experiments are independently repeated 20 times. In this work, a single Query refers to one ground-truth evaluation of a candidate architecture, that is, retrieving its pre-recorded accuracy under the fixed training configuration from the benchmark dataset, which semantically corresponds to a full "train + validate" procedure in a real NAS scenario. Queries thus measure the number of candidate architectures that are actually evaluated during the search and reflect the sample efficiency of the algorithm, rather than the number of internal LLM inference calls.

Among the four variants considered, Reg-PKB achieves the best performance, outperforming all other ENAS-based algorithms with an average accuracy of 94.19%. Both Reg-NoPKB and Cls-PKB exhibit competitive results comparable to the current SOTA methods, in terms of both query efficiency and accuracy. In contrast, Cls-NoPKB fails to reach the SOTA level. Detailed analysis reveals that employing LLMs as classifiers within the ENAS framework is more challenging, particularly in the absence of PKB enhancement, which can lead to hallucination phenomena. Further discussion can be found in Sections 4.5.1 and 4.5.2. As illustrated in Fig. 4(a), in NAS-Bench-101, using LLMs as regression-based surrogate models generally outperforms their use as classifiers, with PKB-enhanced regression-based surrogate models delivering the most accurate performance estimations.

From the results presented in Table 3, it can be observed that all pairwise comparisons reach statistical significance at the 0.05 level. The t-tests indicate that these differences are not attributable to random variation but are statistically meaningful. This finding demonstrates that, in the NAS-Bench-101 experiments, the introduction of the PKB yields stable and reproducible performance improvements. Moreover, the regression-based surrogate model consistently outperforms the classification-based surrogate model, and this superiority is also supported by statistically significant evidence. Overall, the significance analysis further confirms that both the LLM prediction paradigm and the PKB-guided mechanism have substantial impacts on the final search quality within the ENAS framework.

Table 3
Paired T-Test results among different LLM-ENAS variants.

Comparison	t-statistic	*p*-value	Significant at $\alpha < 0.05$?
Reg-PKB vs Reg-NoPKB	3.4750	1.5922e-03	Yes
Reg-PKB vs Cls-PKB	4.0018	5.7259e-04	Yes
Reg-NoPKB vs Cls-NoPKB	4.5680	6.7749e-05	Yes
Cls-PKB vs Cls-NoPKB	2.1035	4.2215e-02	Yes

Table 4
Validation and test accuracy for CIFAR-10 and CIFAR-100. Evaluated on NAS-Bench-201 from twenty independent runs. The table shows the mean and standard deviation. "-" indicates unavailable results.

Algorithm	#Queries	CIFAR-10		CIFAR-100	
		Validation(%)	Test(%)	Validation(%)	Test(%)
Oracle	N/A	91.61	94.37	73.49	73.51
DARTS [27]	–	39.77 ± 0.00	54.30 ± 0.00	15.03 ± 0.00	15.61 ± 0.00
ShuffleNet [34]	1000	–	92.49 ± 1.35	–	70.06 ± 1.72
NAR [28]	1000	91.44 ± 0.10	94.33 ± 0.05	72.54 ± 0.44	72.89 ± 0.37
MbML-NAS (GB) [35]	860	–	93.03 ± 0.52	–	70.02 ± 1.17
MbML-NAS (RF) [35]	860	–	93.36 ± 0.20	–	70.33 ± 0.85
Arch2vec [33]	400	91.32 ± 0.42	94.12 ± 0.42	73.13 ± 0.72	73.15 ± 0.78
PRE-NAS [36]	200	91.37 ± 0.28	94.04 ± 0.34	71.95 ± 1.21	72.02 ± 1.22
GENIUS [21]	-	91.07 ± 0.20	93.79 ± 0.09	70.96 ± 0.33	70.91 ± 0.72
LLMatic [20]	-	91.42 ± 0.13	94.26 ± 0.10	71.41 ± 1.44	71.62 ± 1.73
RZ-NAS [18]	-	91.45 ± 0.10	94.24 ± 0.12	73.35 ± 0.14	73.30 ± 0.21
Reg-PKB	**100**	**91.55 ± 0.11**	**94.32 ± 0.10**	**73.42 ± 0.15**	**73.44 ± 0.14**
Reg-NoPKB	100	91.41 ± 0.13	94.20 ± 0.11	73.24 ± 0.18	73.07 ± 0.49
Cls-PKB	100	91.46 ± 0.18	94.19 ± 0.24	73.19 ± 0.27	73.21 ± 0.23
Cls-NoPKB	100	91.28 ± 0.19	94.05 ± 0.20	72.71 ± 0.71	73.01 ± 0.66

4.3. Results on NAS-bench-201

As shown in Table 4, LLM-ENAS-PKB is further compared with several representative NAS algorithms on the NAS-Bench-201 search space, conducting experiments on CIFAR-10 and CIFAR-100. Each experiment is repeated 20 times to mitigate randomness, and the entire search process is based solely on architecture information from the validation set. The table reports the method name, number of queries, and the mean accuracy with standard deviation for the best architectures identified during the search on both the validation and test sets.

The results indicate that the proposed Reg-PKB and Cls-PKB methods achieve optimal or near-optimal performance. In contrast, Reg-NoPKB and Cls-NoPKB exhibit larger performance variance, mainly due to the strict limit of 100 queries. When the query budget is increased to 200 or more, both methods stably reach performance levels comparable to SOTA algorithms. As illustrated in Fig. 4(b) and (c), Reg-PKB consistently outperforms the other three methods in terms of both stability and accuracy. Furthermore, it can be observed that LLM-based methods without PKB enhancement exhibit significant fluctuations in their results.

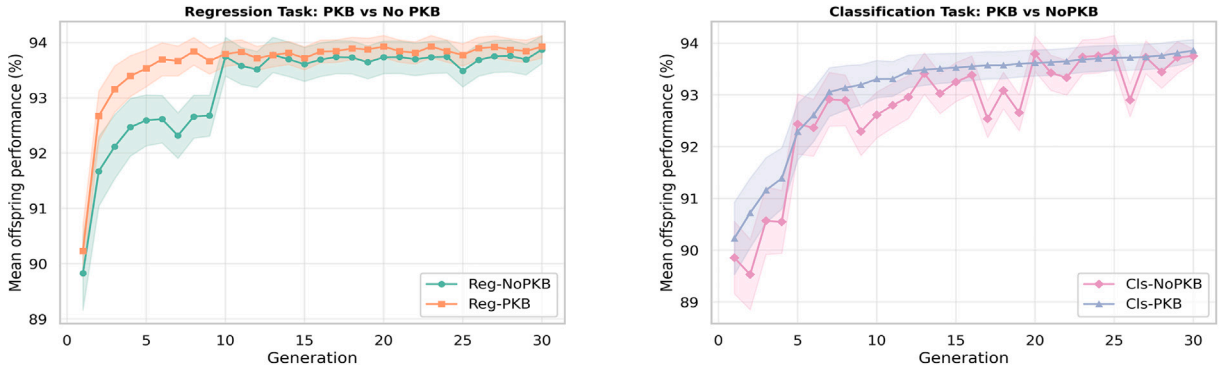
Compared with recent LLM-driven NAS approaches such as LLMatic and RZ-NAS, our method exhibits clear advantages in both stability and efficiency. LLMatic primarily relies on LLM-generated mutation rules to update candidate operations. However, its architecture refinement process remains largely stochastic and sensitive to prompt variations, leading to noticeable fluctuations in accuracy. In contrast, RZ-NAS enhances zero-cost proxies through reflective reasoning, but its reliance on manually designed mutation templates and multi-stage proxy evaluation introduces additional complexity and makes performance highly dependent on the quality of zero-cost metrics. By comparison, the LLM-ENAS-PKB framework provides a more reliable architecture evaluation mechanism. The PKB supplies the LLM with high-quality historical knowledge, effectively suppressing hallucinations and reducing variance, while regression-based and classification-based surrogate models support stable performance estimation even under strict query budgets. As reflected in Table 4, these design choices enable our Reg-PKB variants to match or surpass the performance of LLMatic and RZ-NAS on both CIFAR-10 and CIFAR-100, achieving strong accuracy with significantly fewer queries and markedly improved search stability.

4.4. Results on DARTS

In the large-scale DARTS [27] search space, we constructed a supernetwork and adopted a single-path sampling strategy for pre-training to mitigate the co-adaptation issue among different paths. Within the ENAS framework, the LLM-based regression surrogate model was integrated into the candidate evaluation pipeline: in each generation, LLM-ENAS-PKB first employs the LLM to rapidly filter out promising architectures, after which these candidates inherit the pre-trained supernet weights for fast inference. This process produces high-quality auxiliary supervisory signals for the LLM and improves the reliability of the architectural accuracy stored in the PKB.

Table 5
Comparisons with SOTA algorithms on the DARTS search space.

Method	#Params (M)	Error Rate (%)	Search Cost (GPU days)
ResNet-101 [37]	**1.7**	6.43	–
DenseNet [38]	27.2	3.74	–
DenseNet-BC [38]	25.6	3.48	–
BANANAS [31]	3.6	2.57	11.8
Arch2vec [33]	3.6	**2.48**	10.5
ESENet [39]	4.53	3.56	9
DARTS [27]	3.3	2.76	1.5
LA-DARTS [40]	2.7	2.72	0.7
CARS [41]	3.6	2.62	0.4
LAPT-REA [42]	–	2.65	**0.1**
LLM-ENAS-PKB (ours)	3.7	2.57	0.3



(a) Regression Task: PKB vs No PKB (b)Classification Task: PKB vs No PKB

Fig. 5. On the NAS-Bench-101 platform, a comparison was made regarding whether to use PKB to assist the population iteration process of LLM. The x-axis represents the number of population iterations, and the y-axis represents the mean offspring performance selected by each generation of LLM.

As shown in Table 5, LLM-ENAS-PKB achieves an error rate of 2.57% on CIFAR-10 within the DARTS search space, which is comparable to the best-performing baselines such as BANANAS, and significantly outperforms classical NAS methods including DARTS, ENAS, and GDAS. Notably, the total search cost of our method is only 0.3 GPU days, substantially lower than most reinforcement-learning-based or Bayesian-optimization-based NAS methods, thereby demonstrating the high efficiency of the proposed framework in large-scale search spaces. Compared with existing LLM-based NAS approaches, LLM-ENAS-PKB also exhibits clear advantages. For example, LAPT-REA leverages LLMs to transfer design principles for guiding architectural evolution and reports 2.65% error / 0.1 GPU days. In contrast, LLM-ENAS-PKB achieves a lower error rate while maintaining a similarly lightweight search budget. This indicates that the PKB-enhanced LLM regression-based surrogate model can more accurately capture fine-grained structural differences and offers better predictive stability and generalization ability, especially in large-scale search spaces.

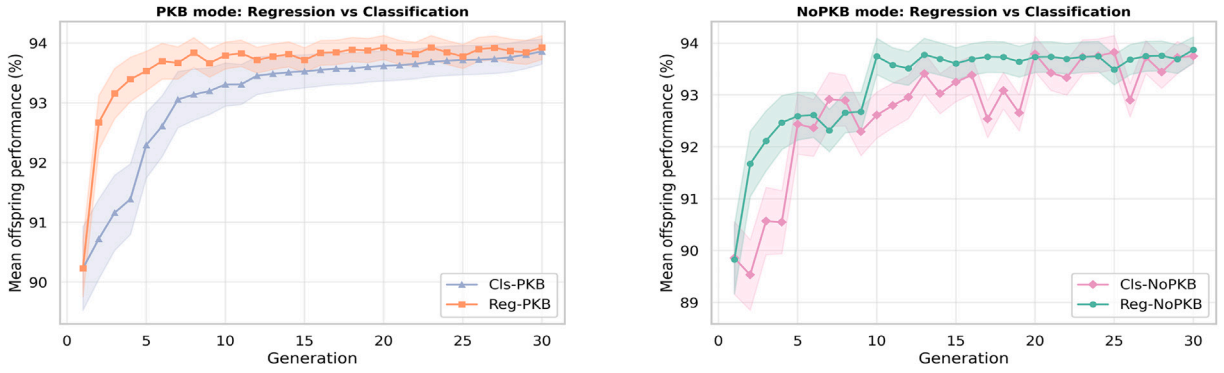
Compared with LAPT-REA, which transfers high-level design principles from LLMs to guide mutation steps, our method benefits from a more fine-grained and data-driven evaluation mechanism. LAPT-REA relies primarily on heuristic architectural refinements inferred from textual knowledge, whereas LLM-ENAS-PKB leverages the PKB-enhanced regression surrogate to capture subtle structural differences, yielding more stable and precise performance estimations. As a result, under a similarly lightweight search cost, our method achieves a lower error rate on CIFAR-10, demonstrating superior predictive reliability and stronger generalization in large-scale search spaces.

In summary, within the highly complex and extremely large DARTS search space, LLM-ENAS-PKB not only achieves competitive accuracy but also greatly reduces the search cost. These results highlight the scalability and practical value of the proposed method, laying a solid foundation for future NAS research on even larger datasets.

4.5. Ablation experiment

4.5.1. Population knowledge base

The PKB aims to enhance the capability of LLMs in utilizing relevant historical knowledge during the neural architecture evaluation stage. The core idea is to select a representative subset of architectures from historical populations that exhibit a balanced performance distribution and rich genetic diversity. This mechanism is introduced because LLMs are typically constrained by a limited context



(a) PKB mode: Regression vs Classification (b) NoPKB mode: Regression vs Classification

Fig. 6. On the NAS-Bench-101 space, a comparison of the population iteration processes for regression and classification tasks executed using LLM. The x-axis represents the number of population iterations, and the y-axis represents the mean offspring performance selected by each generation of LLM.

window, and when the historical information becomes too long, they are prone to hallucinations, forgetting key information, and leading to reasoning confusion, which can impair prediction accuracy and stability.

To verify the effectiveness of PKB in reducing prediction errors, an ablation study is conducted as illustrated in Fig. 5. In this experiment, the LLM is employed as the surrogate model in ENAS-based architecture search, and we compare two settings: with PKB and without PKB. For both settings, we analyze the change in average performance of the population over generations for the architectures selected by the surrogate model.

Fig. 5(a) shows the comparison when the LLM serves as a regression-based surrogate model. With PKB enabled, the population quickly reaches a higher performance level in the early iterations and maintains stable performance with small fluctuations in the later stages. Fig. 5(b) presents the comparison when the LLM is used as a classification-based surrogate model. Without PKB, the average performance of the population exhibits large and frequent oscillations during the evolutionary process.

Such pronounced oscillations indicate that the LLM has a higher probability of making incorrect selections, thereby introducing low-quality architectures into the population. Examination of the LLM outputs reveals that excessive historical architecture information can cause the model to forget the core prompt content or even produce hallucinations. In the context of ENAS, this is high-risk, as low-quality architectures may carry inferior genetic material and disrupt the evolutionary direction in subsequent generations. In contrast, the LLM enhanced with PKB can condense the most relevant information within a shorter context, effectively reducing the probability of erroneous selections. This leads to better stability and robustness, demonstrating that the PKB mechanism can significantly improve the adaptability and reliability of LLMs acting as surrogate models in neural architecture search.

4.5.2. LLM performing regression tasks vs classification tasks

From Tables 2–4, it is evident that the performance of the LLM on regression tasks significantly surpasses its performance on classification tasks. To further validate this observation, we designed and conducted an ablation experiment, as shown in Fig. 6. Regardless of whether PKB is used to enhance the LLM, the results indicate that in the regression task, the quality of the populations selected by the LLM in each generation is consistently higher than when the LLM is used as a classification-based surrogate. Specifically, in the experiment shown in Fig. 6, the LLM serves as the surrogate model during the ENAS architecture evaluation phase. The mean performance of the offspring architectures selected by the surrogate model is recorded during each population iteration. Both Fig. 6(a) and (b) show that the mean performance of the offspring architectures selected for the regression task is consistently higher than that for the classification task across all generations. This observation contrasts with the commonly held belief in traditional machine learning that binary classification tasks are simpler than regression tasks, thus raising a question worth exploring: why does the LLM perform better on regression tasks than binary classification tasks in the context of ENAS?

The reason for the superior performance of the LLM on regression tasks compared to binary classification tasks lies primarily in the inherent differences between regression and classification tasks. Regression tasks require the LLM to predict a continuous performance value, which enables the model to make more fine-grained distinctions between architectures. In regression tasks, the continuous prediction values provide more information, allowing the model to reflect small performance differences between architectures. This finer-grained feedback allows the LLM to learn more effectively, aiding it in making more precise decisions during architecture selection. In contrast, binary classification tasks require the LLM to predict a discrete label (such as 1 or 0), limiting the complexity of the available information and thereby reducing the model's ability to perceive subtle differences between architectures. Specifically, in regression tasks, the LLM can better utilize the feedback signal from the continuous values to identify small performance fluctuations between architectures, which helps optimize architecture selection. However, in classification tasks, since the labels are limited to "high performance" or "low performance", the basis for the LLM's decision-making is more simplistic, thereby limiting its precision in architecture selection.

Thus, regression tasks, by providing continuous and granular performance information, allow the LLM to more accurately identify and select architectures based on subtle differences. On the other hand, classification tasks, due to their discrete nature, make it more

Table 6
Accuracy comparison of different LLMs under the universal template and individually tuned templates. All experiments were conducted on NAS-Bench-101 with 400 queries.

Setting	DeepSeek-R1	GPT-4o	Gemini-2.5-pro	Qwen3
Universal Template (%)	94.19	94.07	94.16	94.11
Individually Tuned Template (%)	94.19	94.08	94.18	94.14

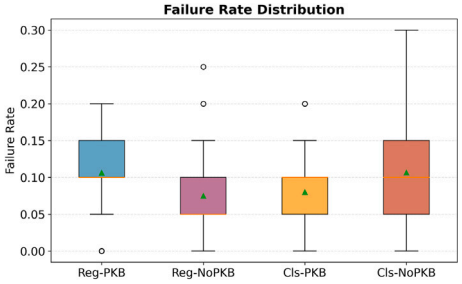


Fig. 7. A comparison of the failure rate distributions across four experimental configurations in which the LLM surrogate is applied to regression and classification tasks, with and without the incorporation of PKB. The x-axis denotes the four settings, and the y-axis represents the per-generation failure rate as defined by the proportion of misranked architectures.

difficult for the LLM to capture small architectural differences, negatively affecting its performance in such tasks. This explains why the LLM performs better in regression tasks than in binary classification tasks within the ENAS framework.

4.5.3. Different choices of LLMs

To further examine the influence of prompt templates on model performance, we evaluate four LLMs using both a universal prompt template (the one optimized primarily using DeepSeek-R1 and shown in Figs. 2 and 3) and individually tuned templates for each model. Table 6 summarizes the results. As shown in Table 6, the accuracy differences between the universal template and the individually tuned templates are minimal across all LLMs (no more than 0.03%). This demonstrates that the universal template already exhibits strong robustness, and its effectiveness is not limited to a specific LLM. Although DeepSeek-R1 achieves the highest score under the universal template, this does not imply inherent superiority. Rather, due to the fact that prompt engineering in this work was primarily performed using DeepSeek-R1, directly transferring this template to other LLMs introduces a tuning bias. The individually tuned template mitigates this bias: after tuning, all LLMs perform nearly identically, and the gap between DeepSeek-R1 and other models becomes even smaller. These observations indicate that (1) our method is highly robust to prompt variations, (2) mild per-model prompt tuning can slightly enhance performance, and (3) the overall conclusions of this work do not depend on a specific LLM backbone. We expect that further task-specific prompt refinement could yield additional performance gains for each model.

4.5.4. LLM prediction robustness analysis

To further assess the reliability of the LLM surrogate beyond average prediction accuracy, we analyze its robustness by introducing the failure rate as an additional evaluation metric. In this study, the failure rate is defined as the proportion of architectures that the LLM incorrectly ranks within the top-20 predicted candidates while their ground-truth validation accuracy lies in the lower 50% of the same generation. This metric highlights high-confidence misjudgments, which pose significant risks in ENAS by potentially directing the evolutionary process toward inferior architectural regions.

Fig. 7 presents the failure rate distributions across four experimental configurations—regression versus classification-based surrogate models, each with and without PKB enhancement. Regression-based surrogate models consistently exhibit lower median failure rates and reduced variability compared to classification-based surrogate models, demonstrating more stable predictive behavior. The incorporation of PKB further reduces both the median failure rate and the inter-quartile range, indicating that PKB provides informative and diverse historical context that mitigates reasoning drift and reduces the likelihood of hallucination-induced misranking. Configurations without PKB, especially in the classification setting, display larger variance and more extreme outliers, suggesting susceptibility to unstable predictions when contextual information is insufficient.

Overall, the analysis shows that regression-based surrogate models offer intrinsically stronger robustness than classification-based surrogate models, and PKB enhances prediction stability across both task types. These observations are consistent with the performance trends reported earlier and reinforce the conclusion that regression-based PKB-enhanced LLM surrogates provide the most reliable evaluation behavior within the ENAS framework.

4.5.5. Hyperparameter analysis

To evaluate the robustness of the PKB selection strategy in Algorithm 2, we conducted a sensitivity study on the hyperparameters α, K_{perf}, K_{div}, and $selected_size$. Overall, the method exhibits stable behavior across a wide range of configurations.

Table 7
Final average performance for different α values.

α	Final Performance (%)
0.2	93.89%
0.4	93.86%
0.6 (ours)	**93.94%**
0.8	93.69%

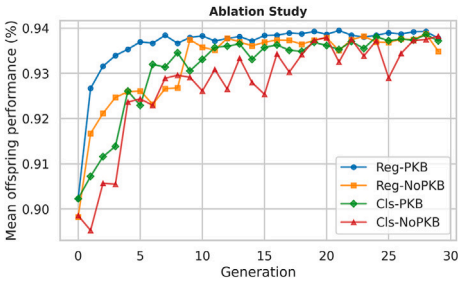


Fig. 8. Performance comparison of four PKB configurations across generations, illustrating the effects of performance-based, diversity-based, and combined selection strategies.

We first examine the weighting coefficient α. As shown in Table 7, varying $\alpha \in \{0.2, 0.4, 0.6, 0.8\}$ produces only minor differences in the final accuracy, with $\alpha = 0.6$ achieving the best performance. This indicates that the performance–diversity combined score is not overly sensitive to the exact choice of α, and a moderate performance-oriented weighting yields slightly better stability.

The parameter $selected_size$ controls the number of historical architectures included in the PKB. Within the effective context capacity of the LLM, larger values provide richer structural–performance patterns and enhance prediction stability. However, because DeepSeek-R1 operates under a practical context-length limit of 128K tokens, the total prompt—including PKB entries, candidate architectures, and instruction text—must remain within this usable window. Since each PKB entry occupies several hundred tokens, an excessively large $selected_size$ may cause the prompt to exceed this limit, leading to an abrupt degradation in prediction quality. Thus, when $selected_size$ surpasses the model's effective context capacity, the prediction behavior deteriorates sharply, implying a hard upper bound rather than a smooth sensitivity trend.

We further analyze the contributions of K_{perf} and K_{div} by comparing four PKB configurations: Reg-PKB (combined selection), Reg-PerfPKB (performance only), Reg-DivPKB (diversity only), and Reg-NoPKB (disabled). As illustrated in Fig. 8, the combined Reg-PKB strategy achieves the highest mean offspring performance throughout the evolution process. The performance-only and diversity-only variants show partial improvements but remain inferior to the full strategy, while disabling the PKB results in pronounced fluctuations and slower convergence. These results confirm that performance and diversity provide complementary information, and their joint use is essential for constructing an informative and stable PKB that effectively supports LLM-based surrogate prediction.

5. Conclusion and future work

This study proposes the LLM-ENAS-PKB framework, which integrates large language models into evolutionary neural architecture search and employs a population knowledge base to provide structured and representative historical information. This design enhances the stability and accuracy of architecture performance prediction. Experimental results on both the NAS-Bench series and the large-scale DARTS search space demonstrate that the framework delivers high search efficiency and high-quality results, confirming the feasibility of using LLMs as lightweight and effective surrogate models. In particular, the regression-based variant with PKB support exhibits consistently stable performance and reliable convergence, showing that the proposed knowledge-enhanced reasoning mechanism is effective for architecture evaluation in the considered settings.

Despite these promising outcomes, several limitations remain. Although the method has demonstrated scalability across multiple representative and large search spaces, its robustness in broader real-world tasks with substantially different data distributions and architectural patterns still requires further investigation, especially beyond the vision domain. Moreover, LLM inference is inherently constrained by context length, susceptibility to redundant information, and the potential risk of hallucination. Although the PKB alleviates these issues to some extent, maintaining fully consistent prediction behavior remains challenging. In addition, the decision-making process of LLMs lacks interpretability, and the current framework does not explicitly reveal which structural patterns the model attends to, which may limit its applicability in scenarios requiring high transparency.

In future work, we aim to strengthen the adaptability of the framework across diverse and cross-domain tasks, extending it from image classification to more complex or multimodal architecture design problems. Further efforts will explore more efficient mechanisms for knowledge organization and prompt compression to mitigate context limitations and reduce hallucination risks. Another promising direction is the development of hybrid evaluation strategies that combine LLM-based surrogate models with lightweight traditional surrogate models, achieving a more favorable balance among computational cost, prediction stability, and interpretability. Overall, our results indicate that integrating large language models into neural architecture search is feasible and effective, and they provide a basis for further studies on automated architecture design.

CRediT authorship contribution statement

Weilin Fang: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation. **Yu Xue:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Funding acquisition. **Lilian Yuan:** Writing – review & editing, Writing – original draft, Software, Methodology, Investigation. **Mohammad Kamrul Hasan:** Writing – review & editing, Supervision, Project administration. **Khursheed Aurangzeb:** Writing – review & editing, Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (NO. 62376127, NO. 61876089, NO. 61876185, NO. 61902281, NO. 61403206), by the Natural Science Foundation of Jiangsu Province (NO. BK20141005), and by the Natural Science Foundation of the Jiangsu Higher Education Institutions of China (NO. 14KJB520025).

Data availability

Data will be made available on request.

References

[1] Y. Chen, F. Liu, K. Pei, Cross-modal matching CNN for autonomous driving sensor data monitoring, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2021, pp. 3110–3119.
[2] M. Zhang, R. Liu, D. Deguchi, H. Murase, Texture-guided transfer learning for low-quality face recognition, IEEE Trans. Image Process. 33 (2023) 95–107.
[3] B. Zoph, Q.V. Le, Neural architecture search with reinforcement learning, in: Proceedings of the International Conference on Learning Representations, 2017.
[4] E. Real, S. Moore, A. Selle, S. Saxena, Y.L. Suematsu, J. Tan, Q.V. Le, A. Kurakin, Large-scale evolution of image classifiers, in: Proceedings of the International Conference on Machine Learning, 2017, pp. 2902–2911.
[5] G. Kadlecov'a, J. Lukasik, M. Pil'at, P. Vidnerov'a, M. Safari, R. Neruda, F. Hutter, Surprisingly strong performance prediction with neural graph features, in: Proceedings of the International Conference on Machine Learning, 2024.
[6] X. Xiang, K. Jing, J. Xu, A neural architecture predictor based on GNN-enhanced transformer, in: Proceedings of the International Conference on Artificial Intelligence and Statistics, 2024, pp. 1729–1737.
[7] J. Jeon, Y. Oh, J. Lee, D. Baek, D. Kim, C. Eom, B. Ham, Subnet-aware dynamic supernet training for neural architecture search, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2025, pp. 30137–30146.
[8] Y. Oh, H. Lee, B. Ham, Efficient few-shot neural architecture search by counting the number of nonlinear functions, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, 2025, pp. 19740–19748.
[9] J. Lee, B. Ham, AZ-NAS: assembling zero-cost proxies for network architecture search, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 5893–5903.
[10] S. Aras, E. Aras, E. Gedikli, H.T. Kahraman, Multi-objective CNN optimization: a robust framework for automated model design, Inf. Sci. 719 (2025) 122468.
[11] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P.J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, J. Mach. Learn. Res. 21 (140) (2020) 1–67.
[12] X. Liao, C. Chen, Z. Wang, Y. Liu, T. Wang, L. Cheng, Large language model assisted fine-grained knowledge graph construction for robotic fault diagnosis, Adv. Eng. Inform. 65 (2025) 103134.
[13] Y. Xue, B. Hu, F. Neri, A surrogate model with multiple comparisons and semi-online learning for evolutionary neural architecture search, IEEE Trans. Emerg. Top. Comput. Intell. (2025) 1–13, https://doi.org/10.1109/TETCI.2025.3547621
[14] Y. Xue, Z. Zhang, F. Neri, Similarity surrogate-assisted evolutionary neural architecture search with dual encoding strategy, Electr. Res. Arch. 32 (2) (2024) 1017–1043.
[15] J. Huang, B. Xue, Y. Sun, M. Zhang, G.G. Yen, Split-level evolutionary neural architecture search with elite weight inheritance, IEEE Trans. Neural Netw. Learn. Syst. 35 (10) (2024) 13523–13537.
[16] C. Yang, X. Wang, Y. Lu, H. Liu, Q.V. Le, D. Zhou, X. Chen, Large language models as optimizers, in: Proceedings of the Twelfth International Conference on Learning Representations, 2023.
[17] Z. Cai, Y. Tang, Y. Lai, H. Wang, Z. Chen, H. Chen, SEKI: Self-evolution and knowledge inspiration based neural architecture search via large language models, arXiv preprint arXiv:2502.20422, 2025.
[18] Z. Ji, G. Zhu, C. Yuan, Y. Huang, RZ-NAS: enhancing LLM-guided neural architecture search via reflective zero-cost strategy, in: Forty-Second International Conference on Machine Learning, 2025.
[19] X. Xue, H. Zhou, T. Chen, S. Zhang, Y. Long, J. Li, Instructing the architecture search for spatial-temporal sequence forecasting with LLM, arXiv preprint arXiv:2503.17994, 2025.
[20] M.U. Nasir, S. Earle, J. Togelius, S. James, C. Cleghorn, LLMatic: neural architecture search via large language models and quality diversity optimization, in: Proceedings of the Genetic and Evolutionary Computation Conference, 2024, pp. 1110–1118.
[21] M. Zheng, X. Su, S. You, F. Wang, C. Qian, C. Xu, S. Albanie, Can GPT-4 perform neural architecture search? arXiv preprint arXiv:2304.10970, 2023.
[22] P. Trirat, J.-G. Lee, Monaq: multi-objective neural architecture querying for time-series analysis on resource-constrained devices, in: Findings of the Association for Computational Linguistics, 2025, pp. 16922–16950.
[23] H. Xu, X. Su, A. Sowmya, I. Katz, D. Wang, SCD-NAS: towards zero-cost training in melanoma diagnosis, in: Proceedings of the IEEE International Conference on Multimedia and Expo, 2024, pp. 1–6.
[24] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, Adv. Neural Inf. Process. Syst. 33 (2020) 9459–9474.
[25] C. Ying, A. Klein, E. Christiansen, E. Real, K. Murphy, F. Hutter, NAS-bench-101: towards reproducible neural architecture search, in: Proceedings of the International Conference on Machine Learning, 2019, pp. 7105–7114.
[26] X. Dong, Y. Yang, NAS-Bench-201: Extending the scope of reproducible neural architecture search, arXiv preprint arXiv:2001.00326, 2020.
[27] H. Liu, K. Simonyan, Y. Yang, DARTS: Differentiable architecture search, arXiv preprint arXiv:1806.09055, 2018.
[28] B. Guo, T. Chen, S. He, H. Liu, L. Xu, P. Ye, J. Chen, Generalized global ranking-aware neural architecture ranker for efficient image classifier search, in: Proceedings of the ACM International Conference on Multimedia, 2022, pp. 3730–3741.

[29] L. Wang, S. Xie, T. Li, R. Fonseca, Y. Tian, Sample-efficient neural architecture search by learning actions for monte carlo tree search, IEEE Trans. Pattern Anal. Mach. Intell. 44 (9) (2021) 5503–5515.
[30] H. Shi, R. Pi, H. Xu, Z. Li, J. Kwok, T. Zhang, Bridging the gap between sample-based and one-shot neural architecture search with BONAS, Adv. Neural Inf. Process. Syst. 33 (2020) 1808–1819.
[31] C. White, W. Neiswanger, Y. Savani, BANANAS: Bayesian optimization with neural architectures for neural architecture search, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, 2021, pp. 10293–10301.
[32] Y. Zhang, B. Li, Y. Tan, S. Jian, Evaluation ranking is more important for NAS, in: Proceedings of the International Joint Conference on Neural Networks, 2022, pp. 1–8.
[33] S. Yan, Y. Zheng, W. Ao, X. Zeng, M. Zhang, Does unsupervised architecture representation learning help neural architecture search? Adv. Neural Inf. Process. Syst. 33 (2020) 12486–12498.
[34] R. Zhang, P.-Y. Zhang, M.-R. Gao, J.-Z. Ma, L.-H. Pan, Low-cost architecture performance evaluation strategy based on pixel difference degree contrast measurement, Appl. Soft Comput. 155 (2024) 111440.
[35] G.T. Pereira, I.B.A. Santos, L.P.F. Garcia, T. Urruty, M. Visani, A.C.P.L.F. de Carvalho, Neural architecture search with interpretable meta-features and fast predictors, Inf. Sci. 649 (2023) 119642.
[36] Y. Peng, A. Song, V. Ciesielski, H.M. Fayek, X. Chang, PRE-NAS: evolutionary neural architecture search with predictor, IEEE Trans. Evol. Comput. 27 (1) (2022) 26–36.
[37] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
[38] G. Huang, Z. Liu, L. Van Der Maaten, K.Q. Weinberger, Densely connected convolutional networks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 4700–4708.
[39] Z. Qiu, W. Bi, D. Xu, H. Guo, H. Ge, Y. Liang, H.P. Lee, C. Wu, Efficient self-learning evolutionary neural architecture search, Appl. Soft Comput. 146 (2023) 110671.
[40] Y. Xu, L. Xie, X. Zhang, X. Chen, B. Shi, Q. Tian, H. Xiong, Latency-aware differentiable neural architecture search, arXiv preprint arXiv:2001.06392, 2020.
[41] Z. Yang, Y. Wang, X. Chen, B. Shi, C. Xu, C. Xu, Q. Tian, C. Xu, CARS: continuous evolution for efficient neural architecture search, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 1829–1838.
[42] X. Zhou, X. Wu, L. Feng, Z. Lu, K.C. Tan, Design principle transfer in neural architecture search via large language models, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, 2025, pp. 23000–23008.